

Xeter — Architecture Documentation

arc42 Template | Version 0.5 — Draft Audience: Developers

Table of Contents

- 1. Introduction and Goals
 - 2. Architecture Constraints
 - 3. System Scope and Context
 - 4. Solution Strategy
 - 5. Building Block View
 - 6. Runtime View
 - 7. Deployment View
 - 8. Cross-cutting Concepts
 - 9. Architecture Decisions
 - 10. Quality Requirements
 - 11. Risks and Technical Debt
 - 12. Glossary
-

1. Introduction and Goals

1.1 Requirements Overview

Xeter is a B2B SaaS observability platform for AI agents. It ingests OpenTelemetry spans emitted by instrumented agent code via an SDK, applies heuristic analysis at runtime to flag anomalous spans, and exposes a dashboard from which users can trigger LLM-powered diagnostics on individual traces.

The primary focus of the first version is detection of tool-call anomalies: wrong tool invoked relative to the prompt, no tool invoked when one was expected, or excessive tool calls.

1.2 Quality Goals

Priority	Quality Goal	Motivation
1	Correctness of flagging	False positives erode trust and will be ignored
2	Low-latency ingestion	Agents run continuously; the pipeline must not become a bottleneck
3	Async analysis	Flagging must not block span storage

Priority	Quality Goal	Motivation
4	Configurability	LLM backend for diagnostics must be swappable (external or local)
5	Multi-tenancy	B2B SaaS requires strict tenant isolation

1.3 Stakeholders

Role	Expectation
B2B customer (developer)	Integrate SDK, view flagged spans, trigger diagnostics
Xeter operator	Deploy and operate the SaaS platform

2. Architecture Constraints

Constraint	Rationale
SaaS deployment only	No on-premise distribution in scope
Single database	Architectural decision; technology TBD
LLM for diagnostics is configurable	Customers may require local models for data sovereignty
SDK languages: Python (primary), TypeScript (secondary)	Python covers ~80% of agent implementations; TypeScript covers fast-growing JS/TS agent ecosystem
View has no business logic	All logic lives in backend services
Multi-tenant	Each B2B customer is an isolated tenant

All architectural constraints resolved. No remaining TBDs in this section.

3. System Scope and Context

3.1 Business Context



LLM Provider (ext/local)
[configurable per tenant]

External interfaces:

Interface	Direction	Description
SDK	Inbound	Customer instruments their agent code; spans sent via OTel
Dashboard (View)	Bidirectional	Developer browses spans, triggers diagnostics
LLM Provider	Outbound	Diagnostics calls external or local LLM

3.2 Technical Context

Channel	Protocol	Notes
SDK → Analyser	OTel (OTLP)	gRPC or HTTP/protobuf — TBD
Analyser → Storage	Internal	TBD
Storage → Presenter	Internal	TBD
Presenter → View	REST (HTTPS) + SSE	REST for queries and actions; SSE for push events (flag update, diagnostic completion)
Diagnostics → LLM	HTTP (OpenAI-compatible or local)	Configurable

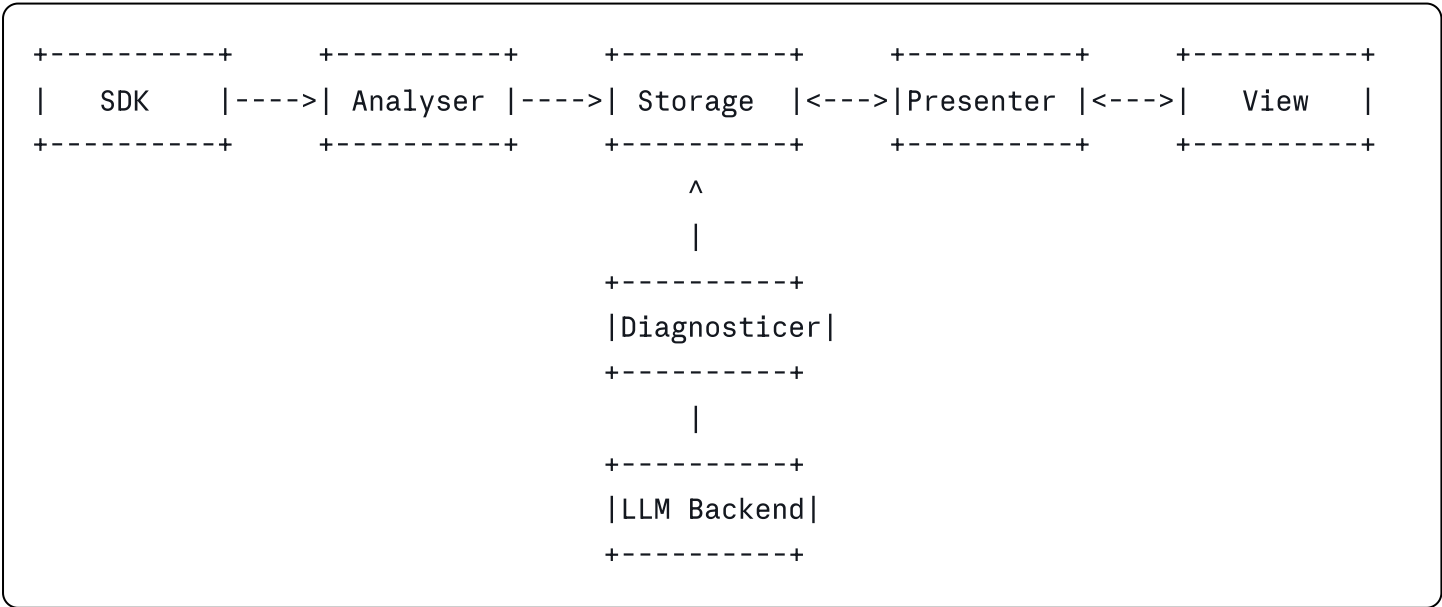
4. Solution Strategy

Problem	Decision	Rationale
Anomaly detection without blocking ingestion	Async flagging pipeline	Span is stored immediately; Analyser runs after
Tool-call anomaly detection	Vector embedding similarity	Compare prompt↔tool_name, prompt↔tool_description, prompt↔response
Parsing error detection	Embedding of model_name + prompt	Detects prompt/model mismatch patterns

Problem	Decision	Rationale
Deep diagnosis	On-demand LLM call from dashboard	Expensive; only run when explicitly requested
High-volume immutable span storage	ClickHouse	OLAP, append-only, proven at scale for this workload by competitors
Mutable relational data (flags, diagnostics, tenants, auth)	PostgreSQL	Low volume, relational, requires inserts and updates
Large text payloads (prompt, response, raw_response)	S3	Prevent row bloat in ClickHouse; store S3 reference key in span row
Ingestion queue for async flagging	Redis	Standard queue for worker pattern; decouples ingestion from embedding
Tenant isolation	Row-level tenant_id on all tables	Scoped at query level in both ClickHouse and PostgreSQL
Cross-store span view assembly	Presenter merges two queries	ClickHouse span + PostgreSQL flags/diagnostics joined in application code; small latency accepted

5. Building Block View

5.1 Level 1 — System Decomposition



5.2 Component Descriptions

SDK

- Wraps OTel instrumentation
- Captures span fields defined in the data model
- Emits spans to the Analyser endpoint
- Language: TBD

Analyser

- Receives spans from SDK
- Stores span immediately (synchronous path)
- Enqueues span for async heuristic analysis
- Heuristics (async):
 - Embed `prompt` and compare to `tool_name`
 - Embed `prompt` and compare to `tool_description`
 - Embed `prompt` and compare to `response`
 - Embed `model_name + prompt` to detect potential parsing errors
 - Classify: no tool called when expected, wrong tool, excessive tool calls
- Writes `flags` JSON to Storage

Storage

- Single database (technology TBD)
- Persists all span fields including `flags` and `diagnostics`
- Must support: tenant isolation, trace reconstruction (tree via `parent_span_id`), append of `flags` and `diagnostics` post-insert

Presenter

- Backend API consumed by View
- Exposes: span list, trace tree, individual span detail
- Routes diagnostic requests to Diagnosticer
- Protocol: TBD (REST / GraphQL / WebSocket)

Diagnosticer

- Separate service
- Triggered by user action via Presenter

- Reads full trace from Storage
- Calls configured LLM with trace context
- Writes diagnostics JSON back to Storage
- LLM backend: configurable (external API or local model, OpenAI-compatible interface assumed)

View

- Frontend only, no business logic
 - Displays span list, trace tree, flags, diagnostics
 - Allows user to select a span/trace and trigger diagnostics
-

6. Runtime View

6.1 Data Flow — Span Ingestion and Flagging

```

Agent Code
|
| (OTel span)
v
Analyser
|-- (sync) --> Storage: INSERT span (flags=null, diagnostics=null)
|-- (async) -> Embedding computation
                    |
                    +--> compare prompt <-> tool_name
                    +--> compare prompt <-> tool_description
                    +--> compare prompt <-> response
                    +--> compare model_name+prompt (parsing errors)
                    |
                    v
                Build flag rows
                    |
                    v
                PostgreSQL: INSERT INTO flags (span_id, flag_type, score, detail,
...)
```

6.2 Diagnostic Flow — User-Triggered

```

View
|
| (user selects trace, clicks "Diagnose")
v
```

```

Presenter
|
|--> ClickHouse: fetch all spans for trace_id
|--> PostgreSQL: fetch all flags for trace_id
|
|--> Diagnosticer
    |
    |--> LLM Backend (configured: external or local)
    |     prompt = trace context + flags
    |     response = diagnostic analysis
    |
    |--> PostgreSQL: INSERT INTO diagnostics (span_id, llm_backend, result,
... )
    |
    v
Presenter: fetch updated diagnostics from PostgreSQL
|
v
View (merged span + flags + diagnostics)

```

7. Deployment View

TBD. SaaS target confirmed. Storage stack decided (see AD-07, AD-12, AD-13). The following are deferred decisions:

- Cloud provider
- Containerisation / orchestration strategy
- Ingestion queue sizing and technology (Redis confirmed; topology TBD)
- CDN / edge for View
- LLM proxy or gateway for local model routing

8. Cross-cutting Concepts

8.1 Multi-tenancy

Every span is associated with a tenant. All Storage queries must be scoped by tenant ID. SDK authentication ties a session to a tenant. Details TBD pending auth decision.

8.2 Authentication and Authorization

Two auth surfaces with different mechanisms.

Surface 1 — SDK → Analyser (ingestion): API key per tenant. Key hash stored in PostgreSQL `api_keys` table. Validated on every ingestion request. Stateless and non-blocking.

Surface 2 — User → Dashboard: Two parallel implementation paths, schema-compatible:

- **Path A (initial, free):** Email + password. Credentials stored in PostgreSQL `users` table (`password_hash`). No external dependency.
- **Path B (future):** Clerk. Handles multi-member per tenant, SSO/SAML for enterprise customers. Migration requires populating `clerk_user_id` and nulling `password_hash` — no schema surgery.

Multi-member support is built into the schema from day one: multiple `users` rows per `tenant_id`. No roles — Xeter is a logger, not a permission-sensitive system.

8.3 Data Model

Three stores. Spans are immutable once written. Flags and diagnostics are separate append-only tables in PostgreSQL, referenced by `span_id`. The Presenter merges results from both stores at read time.

ClickHouse — `spans` table

Every span is written here regardless of whether it is flagged. Rows are immutable after insert.

Field	Type	Notes
<code>tenant_id</code>	string	Partition key. Required for tenant isolation
<code>trace_id</code>	string	Groups spans into one session
<code>span_id</code>	string	Unique per row. Primary key
<code>parent_span_id</code>	string / null	Null if root span. Multiple roots per trace allowed
<code>time_begin</code>	timestamp	
<code>time_end</code>	timestamp	
<code>agent_name</code>	string	
<code>agent_model</code>	string	Set by SDK or inferred
<code>recipient</code>	string	<code>"user"</code> or agent name
<code>recipient_model</code>	string / null	Null if recipient is user
<code>tool_name</code>	string / null	Null if no tool call in this span
<code>tool_description</code>	string / null	Null if no tool call
<code>tool_output</code>	string / null	Null if no tool call
<code>prompt_ref</code>	string	S3 key for prompt payload
<code>response_ref</code>	string	S3 key for agent-processed response
<code>raw_response_ref</code>	string	S3 key for original LLM output

Large text fields (`prompt`, `response`, `raw_response`) are stored in S3. The `_ref` columns hold the S3 object key. The Presenter fetches S3 content on demand when a user opens a span detail view.

PostgreSQL — `flags` table

Append-only. One row per flag per span. A span with no anomalies has no rows here.

Field	Type	Notes
<code>flag_id</code>	uuid	Primary key
<code>tenant_id</code>	string	For scoped queries
<code>span_id</code>	string	Foreign key → ClickHouse spans
<code>trace_id</code>	string	Denormalised for trace-level queries
<code>flag_type</code>	string	e.g. <code>wrong_tool</code> , <code>no_tool</code> , <code>excessive_tool</code> , <code>parsing_error</code>
<code>score</code>	float	Similarity score or confidence from embedding comparison
<code>detail</code>	JSON	Context: which fields were compared, distance, etc.
<code>created_at</code>	timestamp	

PostgreSQL — `diagnostics` table

Append-only. One row per diagnostic run per span. A span may have multiple diagnostics if the user triggers analysis more than once.

Field	Type	Notes
<code>diagnostic_id</code>	uuid	Primary key
<code>tenant_id</code>	string	For scoped queries
<code>span_id</code>	string	Foreign key → ClickHouse spans
<code>trace_id</code>	string	Denormalised for trace-level queries
<code>llm_backend</code>	string	Which LLM was used (model name or endpoint)
<code>result</code>	JSON	Diagnostic output from LLM. Structure TBD
<code>created_at</code>	timestamp	

PostgreSQL — `tenants` table

Field	Type	Notes
<code>tenant_id</code>	uuid	Primary key
<code>name</code>	string	Company or account name
<code>created_at</code>	timestamp	

PostgreSQL — `users` table

Field	Type	Notes
<code>user_id</code>	uuid	Primary key
<code>tenant_id</code>	uuid	FK → tenants. Multiple rows per tenant allowed from day one
<code>email</code>	string	Unique
<code>password_hash</code>	string / null	Path A only. Null when migrated to Clerk
<code>clerk_user_id</code>	string / null	Path B only. Populated on Clerk migration
<code>created_at</code>	timestamp	

PostgreSQL — `api_keys` table

Field	Type	Notes
<code>key_id</code>	uuid	Primary key
<code>tenant_id</code>	uuid	FK → tenants
<code>key_hash</code>	string	Hash only — plaintext never stored
<code>created_at</code>	timestamp	

Open questions:

- `diagnostics.result` JSON structure: typed fields or free-form text?
- `agent_model` / `recipient_model` inference logic: where does it live (SDK or Analyser)?
- S3 payload retrieval: eager (always fetch on span load) or lazy (only on detail view)? Lazy is strongly preferred for list views.

8.4 Embedding Strategy

- Tool-call anomaly detection uses vector similarity between pairs of fields
- Reference corpus: none defined yet — embeddings compare fields within the same span
- Similarity threshold for flagging: TBD (requires calibration on real data)
- Embedding model: TBD

8.5 Error Handling

- If Analyser embedding fails: span already stored in ClickHouse; no flag row written; failure logged
- If Diagnosticer LLM call fails: error surfaced to View; no diagnostic row written; existing data unaffected
- If ClickHouse span insert fails: span lost; SDK must retry or accept loss (TBD — depends on whether ingestion queue provides durability)
- If PostgreSQL flag/diagnostic insert fails: span exists but is unflagged/undiagnosed; retry from queue is possible since span is idempotent in ClickHouse

8.6 Observability of Xeter Itself

TBD. Ironic but necessary.

9. Architecture Decisions

ID	Decision	Status	Rationale
AD-01	Async flagging	Decided	Span ingestion must not be blocked by embedding computation
AD-02	Diagnosticer as separate service	Decided	LLM calls are slow and on-demand; isolation allows independent scaling
AD-03	Split storage (ClickHouse + PostgreSQL + S3)	Decided	Replaces single-database decision; see AD-07, AD-12, AD-13
AD-04	View has no logic	Decided	All logic centralised in backend
AD-05	LLM backend configurable	Decided	Data sovereignty requirements in B2B

ID	Decision	Status	Rationale
AD-06	Multi-tenant architecture	Decided	B2B SaaS requirement; <code>tenant_id</code> on all tables
AD-07	Span storage: ClickHouse	Decided	High-volume, append-only, immutable rows; OLAP queries; proven by Langfuse and LangSmith at identical workload
AD-08	Flags and diagnostics storage: PostgreSQL	Decided	Low-volume, mutable, relational; append-only rows referencing <code>span_id</code>
AD-09	Large payload storage: S3	Decided	<code>prompt</code> , <code>response</code> , <code>raw_response</code> stored in S3; ClickHouse holds reference key only
AD-10	Spans are immutable; flags/diagnostics are separate tables	Decided	Eliminates ClickHouse mutation cost; append-only pattern on both stores
AD-11	Presenter merges ClickHouse + PostgreSQL at read time	Decided	Two queries per span view; small latency accepted
AD-12	Ingestion queue: Redis	Decided	Decouples ingestion from embedding worker; standard pattern
AD-13	Presenter protocol: REST + SSE	Decided	REST for all requests (span list, detail, diagnostic trigger); Server-Sent Events for server-push of two specific events: flag update on span, diagnostic completion. Avoids polling overhead without WebSocket complexity.
AD-14	Auth mechanism: self-hosted email/password → Clerk	Decided	Two parallel paths: Path A (free/early) uses email + password with hashed credentials in PostgreSQL, no external dependency; Path B migrates to Clerk for multi-member, SSO, enterprise. Schema designed for zero-surgery migration between paths. API keys for SDK ingestion in both paths.
AD-15	SDK primary language: Python	Decided	Python powers ~80% of AI agent implementations; strongest embedding library ecosystem (sentence-transformers, FAISS); matches backend language
AD-16	SDK secondary language: TypeScript	Decided	Fast-growing agent ecosystem (LangChain.js, OpenAI Agents SDK, Vercel AI SDK); covers Codex/Claude coding agent use cases; required for B2B coverage

ID	Decision	Status	Rationale
AD-17	Backend services language: Python	Decided	Analyser requires embedding libraries that are Python-native; backend is internal, not customer-facing; aligns with team comfort
AD-18	TypeScript SDK will lag Python SDK by one release cycle	Decided	Single maintainer; schema changes hit Python first; TypeScript follows; must be documented for customers

10. Quality Requirements

10.1 Quality Scenarios

ID	Quality	Scenario	Expected Response
QS-01	Correctness	Analyser flags a span where prompt is "schedule a meeting" and tool_name is "get_weather"	Flag is written with high anomaly score
QS-02	Correctness	Analyser does not flag a span where tool use matches prompt semantically	No flag written or low score below threshold
QS-03	Performance	1000 spans/second ingested from a customer's agent cluster	Spans stored without delay; flagging completes asynchronously within acceptable window (TBD)
QS-04	Availability	LLM provider is unreachable	Diagnosticer returns error to View; existing data unaffected
QS-05	Isolation	Tenant A queries spans	Tenant B's data is never returned

11. Risks and Technical Debt

ID	Risk	Impact	Mitigation
R-01	ClickHouse operational complexity	Medium — harder to operate than PostgreSQL alone; self-hosted customers would inherit this	Use managed ClickHouse (ClickHouse Cloud) for SaaS; document operational requirements

ID	Risk	Impact	Mitigation
R-02	Cross-store query latency	Low-Medium — two queries per span view adds round-trip overhead	Accepted; Presenter parallelises the two queries where possible
R-03	Embedding threshold uncalibrated	High — too many false positives destroys trust	Requires real agent data for calibration; plan a tuning phase before GA
R-04	Auth deferred	Medium — cannot safely onboard B2B customers without it	Required before any external access
R-05	<code>agent_model</code> / <code>recipient_model</code> inference undefined	Low for now	Define inference logic before SDK implementation
R-06	S3 payload retrieval strategy undefined (eager vs lazy)	Medium — eager retrieval on list views will be slow and expensive	Decide before implementing Presenter; lazy strongly preferred
R-07	<code>diagnostics.result</code> JSON structure undefined	Low — blocks Diagnosticer implementation	Define schema before implementing Diagnosticer
R-08	Dual SDK maintenance (Python + TypeScript)	Medium — every schema or OTel change must be applied twice; drift between SDKs will cause customer bugs	Version lock both SDKs together; write integration tests that run against both

12. Glossary

Term	Definition
Span	A single recorded unit of agent execution, one immutable row in ClickHouse
Trace	A group of spans sharing the same <code>trace_id</code> , representing one agent session
Root span	A span with <code>parent_span_id = null</code> . A trace may have multiple roots
Flag	A row in the PostgreSQL <code>flags</code> table, written by the Analyser, indicating a detected anomaly on a span
Diagnostic	A row in the PostgreSQL <code>diagnostics</code> table, written by the Diagnosticer via LLM analysis, on user request

Term	Definition
Analyser	Service responsible for receiving spans, writing them to ClickHouse, and asynchronously computing flags
Diagnosticer	Separate service called on demand to run LLM-based trace analysis and write diagnostic rows
Presenter	Backend API layer that merges data from ClickHouse, PostgreSQL, and S3 before serving the View
View	Frontend dashboard, no business logic
OTel	OpenTelemetry — the observability framework used by the SDK for span emission
Heuristic	In this context: vector embedding similarity comparison between span fields
raw_response	The original LLM output before any agent-side modification; stored in S3
response	The agent-processed version of raw_response, as actually used; stored in S3
payload_ref	An S3 object key stored in the ClickHouse span row pointing to a large text field

Document status: Draft v0.5. All architecture decisions resolved. Remaining open questions: `diagnostics.result` schema, `agent_model`/`recipient_model` inference ownership, S3 retrieval strategy (eager vs lazy).